

Callbacks

You often need to tweak VIP behavior for one test

Register, enable, and hook order

- The callback base class defines virtual pre and post methods
- Your test registers a derived callback on the host with `uvm_callbacks add`
- The host invokes registered callbacks in registration order at each hook point
- Pre hooks can modify the transaction before drive; post hooks observe after the body runs
- Disable a callback and the host skips it even if still registered
- Typical uses

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Callbacks sketch

INTERACTIVE TOOL

Callbacks sketch

Hook behavior into a component **without subclassing**: register **pre/post** callbacks, enable or disable them, then Drive. Starter: `err_inj` on `pre_drive` already registered and armed.

Starter example: callback `err_inj` is registered on the driver's `pre_drive` hook and enabled — no driver subclass.

Load starter example

Challenges 0 / 22 cleared

Quiz: why: UVM callbacks let you...

☐ hook pre/post behavior without subclassing the host component

☐ replace `connect_phase` entirely

☐ synthesize the DUT

☐ set the simulator timescale

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

host hooks

pre → body → post call sites in the component.

register

Attach a callback object — no subclass required.

enable

Turn a registered callback on or off.

reuse

Inject faults, log, or sample without forking the VIP.

Lab

Callbacks lab starter

learn_uvm2017 — uvm callbacks

Real UVM literacy

```
❯ cat - module18-uvm-callbacks/examples/callbacks-sketch

# Track A -- real Unix
# real shell session (Track A -- UVM callbacks)

$ pwd
/mt/d/proj/designs/learning/courses/learn_uvm2017/module18-uvm-callbacks

$ ls examples/callbacks-sketch/
callbacks.txt

$ cat examples/callbacks-sketch/callbacks.txt
UVM callbacks sketch (literacy)
=====

Ideas:
  extend a host component (driver, monitor, ...) at hook points
  without subclassing the host - register callback objects instead

Hook flow (driver example):
  pre_drive(item) → drive body on vif → post_drive(item)

Callback base (typical):
class my_driver_cb extends uvm_callback;
  virtual function void pre_drive(my_driver drv, my_item item);
  virtual function void post_drive(my_driver drv, my_item item);

Derived callback (test-specific):
class err_inj_cb extends my_driver_cb;
  function void pre_drive(my_driver drv, my_item item);
    item.data = 8'h01; // fault inject before drive
  endfunction

Registration (pair types once):
'uvm_register_cb(my_driver, err_inj_cb)

Add instance on the host:
err_inj_cb cb = err_inj_cb::type_id::create("cb");
uvm_callbacks#(my_driver, err_inj_cb)::add(drv, cb);

Host invokes at hook points:
'uvm_do_callbacks(my_driver, err_inj_cb, pre_drive(this, item))
// ... drive body ...
'uvm_do_callbacks(my_driver, err_inj_cb, post_drive(this, item))

Enable / disable:
uvm_callbacks#(my_driver, err_inj_cb)::display(drv);
cb.set_enabled(0); // registered but skipped on next hook

Why callbacks vs subclass:
V2P driver stays vendor-clean
per-test hooks: err inject, logging, coverage
multiple callbacks - registration order = invocation order

Common mistakes:
forgot 'uvm_register_cb(type pairing)
disabled confused with unregistered
heavy work in pre hook stalls driver
two pre callbacks fighting over same field - mind order

Legacy reference:
learn_uvm2017_sv_verilator/modules/examples/callbacks/callbacks.sv

$ grep -n "uvm_register_cb|uvm_do_callbacks|pre_drive|post_drive"
./../learn_uvm2017_sv_verilator/modules/examples/callbacks/callbacks.sv | head -14
6: * 3. Register callback registration ('uvm_register_cb)
9: * 4. Understand callback invocation ('uvm_do_callbacks)
72: * - pre_drive: Called before driving transaction
73: * - post_drive: Called after driving transaction
76: class DriverCallbackBase extends uvm_callback;
77:   'uvm_object_utils(DriverCallbackBase)
79:   function new(string name = "DriverCallbackBase");
98:   virtual function void pre_drive(uvm_driver #(CallbackTransaction) driver,
129:   virtual function void post_drive(uvm_driver #(CallbackTransaction) driver,
140: * - 'uvm_register_cb: Registers callback type with component type
144: class DriverCallback extends DriverCallbackBase;
145:   'uvm_object_utils(DriverCallback)
152: //   'uvm_register_cb(component_type, callback_type)
156: //   - callback_type: Callback class (DriverCallback)
```

Real shell — callbacks sketch

Pitfalls to watch

- Do not subclass a VIP driver for every test tweak, callbacks keep the base driver stable
- Do not forget to register the callback type with the host type or uvm do callbacks will
- Do not assume disabled means unregistered
- Do not put time-consuming work in pre hooks if it stalls the driver, keep hooks focused
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser
- On real UVM, sketch one pre hook and one post hook on a driver
- When you are ready, take the short quiz, then continue to reporting in the next module